CASE STUDY

Object-Oriented Design Principles in a Social Media Application

A professionally structured design study of how object-oriented analysis, class boundaries, interfaces, and service collaboration can be used to build a scalable social media platform.

Prepared By	Arshdeep Singh
UID	24BCS10184
Section	601-A
Academic Focus	Object-Oriented Design Principles and System Structuring

Executive Abstract

This report studies a hypothetical social media application and shows how core object-oriented design principles make the platform easier to extend, test, and maintain. Rather than mixing every feature inside a single User or Post class, the design distributes responsibilities across domain objects, services, repositories, and strategy interfaces so that new features—such as feed ranking, moderation, privacy controls, or notification channels—can evolve without destabilizing the whole system.

Submitted for academic review

Index

This index page combines the submission details and the section map of the report. It replaces a separate contents page so the front matter stays compact and purposeful.

Student Name	Arshdeep Singh	UID	24BCS10184
Section	601-A	Topic	Object-Oriented Design Principles in a Social Media Application

Case premise: The report assumes a growing platform with profiles, posts, comments, likes, media attachments, feeds, notifications, moderation, and persistence. The aim is not to model every technical detail, but to show how a clean object-oriented structure prevents feature growth from turning into architectural chaos.

1. Case Context and Design Goals

A social media application is one of the clearest real-world examples of why object-oriented design matters. Even a modest product contains many interacting behaviors: users create profiles, publish posts, attach media, react to content, comment in threads, follow one another, receive notifications, and view ranked feeds. At a small scale, a student developer may be tempted to implement all of these features directly inside a few large classes. That approach appears fast at first, but it creates brittle code because every new requirement touches the same objects and the same methods. A change in privacy rules, for example, should not force a rewrite of feed ranking, and a change in notification delivery should not require modifying the Post class.

For this case study, the application is treated as a layered social media platform that must remain understandable while new features are continuously added. The design goal is to assign one clear responsibility to each part of the system and then define how those parts collaborate. In practice, that means stable domain objects for core business meaning, service classes for cross-cutting workflows, and repository interfaces for persistence.

2. Domain Model and Responsibility Allocation

The most important decision is the domain model. A User object should represent identity and social ownership, not every detail of presentation, storage, and feed generation. A separate Profile object can own avatar, bio, preferences, and privacy options. A Post object can own caption, author, visibility, timestamp, and attached media references. Comment and Like should remain small, purpose-specific entities because they evolve for different reasons. Another strong object-oriented choice is the use of composition instead of deep inheritance: a Post can compose a MediaAttachment collection, a VisibilityPolicy, or a ModerationState rather than inheriting artificial behavior from a bulky superclass.

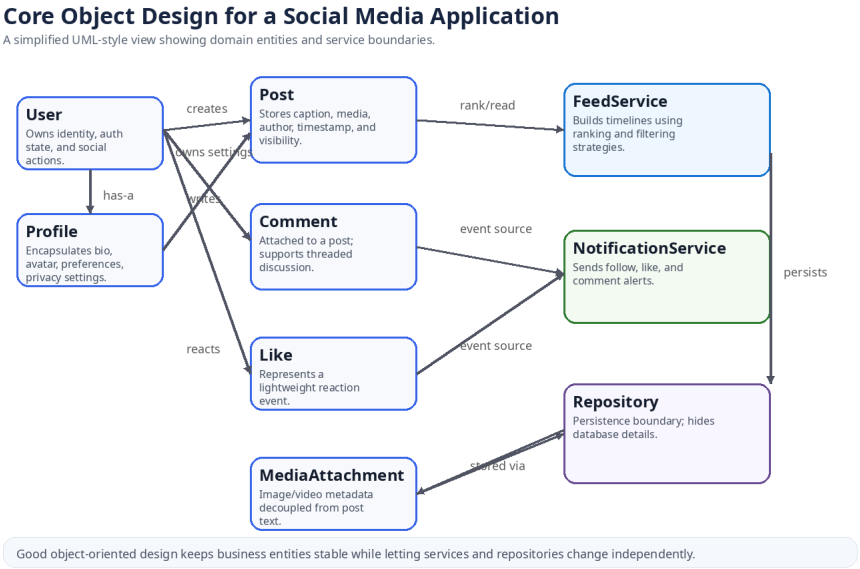

Figure 1. Simplified object view showing domain entities, service components, and the repository boundary.

Figure 1 highlights the separation between business entities and coordination services. User, Profile, Post, Comment, Like, and MediaAttachment hold business meaning. FeedService and NotificationService coordinate workflows that cross entity boundaries. The repository stays behind an abstraction boundary so the domain layer does not become dependent on storage details.

3. Applying OOP and SOLID Principles

The application demonstrates the core object-oriented ideas in a practical way. Encapsulation appears when a Post protects editing and visibility rules and when a Profile controls privacy changes through explicit methods rather than public field mutation. Abstraction appears when controllers interact with interfaces such as FeedReader, PostRepository, or NotificationGateway instead of raw database logic. Polymorphism becomes useful when the system supports multiple ranking strategies, moderation policies, or notification channels through a shared interface.

Component	Primary responsibility	Why it should stay isolated
User / Profile	Identity, account state, preferences, privacy, and social ownership.	Presentation and privacy concerns change faster than core identity rules.
Post	Content, visibility, author link, media references, and publishing rules.	Feed ranking or alert delivery should not be embedded inside content objects.
Comment / Like	Threaded discussion and lightweight reactions.	Reaction logic evolves independently of post editing and profile management.
FeedService	Timeline assembly, ranking, filtering, and recommendation policies.	Ranking changes frequently and benefits from strategy-based extension.
NotificationService	Converts events into user alerts across channels.	Notification delivery is a cross-cutting workflow, not a core entity concern.
Repository interfaces	Persistence boundary and data access contract.	Swapping SQL, cache, or test doubles should not disturb domain rules.

Through the SOLID lens, Single Responsibility discourages “god objects” that mix posting, ranking, persistence, and alerting. Open/Closed encourages extension through new ranking or moderation strategies rather than editing stable entity code. Interface Segregation prevents clients from depending on methods they do not need, and Dependency Inversion keeps high-level workflow code dependent on interfaces rather than concrete SQL classes or third-party SDKs.

4. Interfaces, Patterns, and Interaction Workflow

A strong social media design usually relies on a few classic patterns. The Strategy pattern is useful for feed ranking because chronological ordering, popularity-based ranking, and personalization can all satisfy one interface such as RankStrategy. The Observer or event-driven pattern fits notifications because a new comment, follow, or reaction can emit an event that multiple listeners consume. The Repository pattern hides data access complexity and protects the rest of the code from SQL, cache, or queue details.

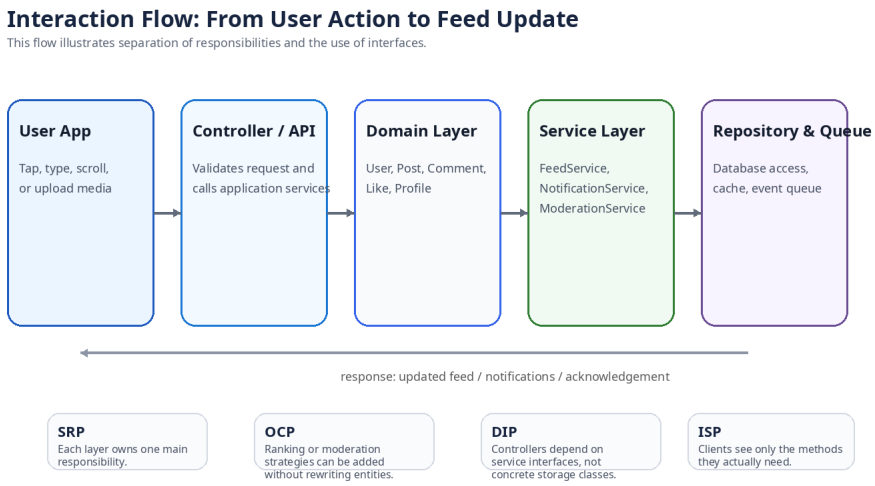

Figure 2. Interaction flow from user action to service execution, persistence, and response generation.

Figure 2 presents a layered request path. The user app triggers an action through a controller or API layer. That layer validates input and delegates the business step to a service. The service then collaborates with domain objects and repository interfaces to complete the operation. The response returns to the client without exposing low-level storage details, which keeps reasoning local and debugging more disciplined.

Pattern / interface	Contribution to the application
RankStrategy	Allows multiple feed ranking policies without rewriting Post or FeedService internals.
NotificationGateway	Supports app alerts, email, or push notifications behind one contract.
Repository + ModerationPolicy	Separates storage and moderation concerns from core domain logic, improving testability.

5. Trade-offs, Risks, and Recommendations

Object-oriented design improves maintainability, but it still demands judgment. Too little structure produces tightly coupled code; too much structure produces unnecessary ceremony. The most practical recommendation is to isolate only the parts of the application that are likely to change independently: ranking logic, notification delivery, moderation policy, privacy rules, and persistence. Stable business entities should remain central, while extension points should be introduced only where product evolution is genuinely expected.

Benefit	Associated trade-off
High cohesion and clearer class responsibilities	Requires discipline in modeling and naming responsibilities correctly.
Easier testing through interfaces and dependency inversion	Adds more files and abstractions than a quick prototype.
Safer extension via strategies and service boundaries	Over-abstraction can slow development if future variation never appears.

6. Conclusion

Object-oriented design principles are highly relevant to a social media application because the problem domain contains many related but independently changing concerns. A well-designed system distinguishes stable domain entities from cross-cutting services, uses interfaces to control dependencies, and relies on carefully chosen patterns to make change predictable rather than disruptive. The core lesson is that good design is not about creating the maximum number of classes; it is about assigning responsibility with precision, preserving boundaries between what changes quickly and what changes slowly, and ensuring that new features can be added by extension rather than by repeatedly rewriting the core model.

Starter architecture	Suggested elements
Domain layer	User, Profile, Post, Comment, Like, MediaAttachment, simple domain rules.
Application layer	FeedService, NotificationService, ModerationService, event handlers, ranking strategies.
Infrastructure layer	Repository implementations, SQL/NoSQL access, cache, queue, push or email gateways.

Final takeaway: Good object-oriented design does not chase complexity for its own sake. It creates a stable core model, introduces extension points only where they are justified, and keeps feature growth controlled instead of chaotic.